

JAFTE CARNEIRO FAGUNDES DA SILVA

**Analizador de Contatos Enron:
Implementação de Algoritmos de Grafos
em Java**

Curitiba

maio de 2026

Sumário

1	INTRODUÇÃO	7
1.1	Motivação	8
2	REFERENCIAL TEÓRICO	9
2.1	Grafos Direcionados, Ponderados e Rotulados	9
2.2	Busca em Profundidade (DFS)	9
2.3	Busca em Largura (BFS)	11
2.4	Algoritmo de Dijkstra	11
2.4.1	Adaptação: Custo Inverso para Maximizar Dependência	11
2.5	Tratamento de Ciclos	12
3	METODOLOGIA E IMPLEMENTAÇÃO	13
3.1	Arquitetura em Camadas	13
3.2	Estrutura de Dados: Grafo com Lista de Adjacência	14
3.3	Algoritmos Implementados	14
3.3.1	1. Busca em Profundidade (DFS)	14
3.3.2	2. Busca em Largura (BFS)	15
3.3.3	3. Distância D (BFS por Níveis)	16
3.3.4	4. Caminho Crítico (Dijkstra Adaptado)	17
3.4	Leitura e Parsing do Dataset	18
4	APRENDIZADOS E CONCLUSÕES	19
4.1	Conceitos Reforçados	19
4.1.1	Estruturas de Dados	19
4.1.2	Algoritmos de Grafos	19

4.1.3	Engenharia de Software	19
4.2	Desafios Enfrentados	20
4.3	Possíveis Extensões	20
4.4	Conclusão	21
A	DIAGRAMAS UML	23
B	EXEMPLOS DE SAÍDA	35
B.1	Modo Demonstração	35
B.2	Consulta BFS no Demo	36

Resumo

Este relatório documenta o desenvolvimento de um analisador de contatos baseado no *Enron Email Dataset* público, implementado em Java 17 como exercício prático da disciplina RESOLUÇÃO DE PROBLEMAS COM GRAFOS. O projeto implementa, do zero, algoritmos clássicos de teoria dos grafos — busca em profundidade (DFS), busca em largura (BFS), cálculo de distância e caminho crítico via Dijkstra adaptado — aplicados a um grafo direcionado, ponderado e rotulado que modela a rede de comunicação entre funcionários da empresa.

A arquitetura segue rigorosamente os princípios SOLID de engenharia de software, com separação clara de responsabilidades em 7 pacotes e 29 classes. Todos os algoritmos tratam ciclos explicitamente via conjuntos de visitados. A interface gráfica, construída com Java Swing, oferece 4 abas de análise interativa com visualização de grafos via GraphStream.

Palavras-chave: Grafos. DFS. BFS. Dijkstra. Java. Enron. Estruturas de Dados.

1 Introdução

A teoria dos grafos é uma das áreas fundamentais da ciência da computação, com aplicações práticas em redes sociais, sistemas de roteamento, análise de dependências e muitas outras áreas. Neste trabalho, desenvolvemos um analisador de contatos baseado no *Enron Email Dataset*, um conjunto de dados públicos contendo mais de 600 mil mensagens de e-mail de aproximadamente 150 funcionários da empresa Enron, coletadas entre 1998 e 2002.

O objetivo do projeto foi consolidar conhecimentos sobre estruturas de dados e algoritmos de grafos através de uma implementação prática completa, desde a representação do grafo até a execução de consultas complexas. Para isso, implementamos:

1. Um grafo direcionado, ponderado e rotulado em memória
2. Busca em profundidade (DFS) iterativa
3. Busca em largura (BFS) com garantia de caminho mínimo
4. Cálculo de distância D entre nós
5. Caminho crítico aproximado via Dijkstra adaptado com custo inverso

Todos os algoritmos foram implementados **do zero**, sem uso de bibliotecas prontas de estruturas de dados ou algoritmos, utilizando apenas Java Collections (HashMap, HashSet, ArrayList, ArrayDeque) e a biblioteca padrão do Java 17.

1.1 Motivação

A análise de redes de comunicação é relevante para entender padrões de colaboração, identificar pessoas-chave em organizações e detectar caminhos críticos de informação. O *Enron Email Dataset* representa um caso de estudo real, permitindo explorar estes tópicos com dados autênticos.

2 Referencial Teórico

2.1 Grafos Direcionados, Ponderados e Rotulados

Um grafo é uma estrutura matemática composta por um conjunto de vértices (ou nós) e um conjunto de arestas (ou arcos) que conectam pares de vértices. Neste projeto, utilizamos:

- **Direcionado:** cada aresta tem uma direção (origem $\rightarrow$ destino). A aresta $A \rightarrow B$ é distinta de $B \rightarrow A$.
- **Ponderado:** cada aresta tem um peso (neste caso, a frequência de mensagens enviadas de origem para destino).
- **Rotulado:** cada vértice tem um rótulo (o endereço de e-mail do indivíduo).

Representação interna: lista de adjacência via mapa de mapas:

```
Map<Vertex, Map<Vertex, Edge>> adjacency
// adjacency.get(origin).get(destination) = Edge com peso
```

Esta representação oferece $O(1)$ lookup de arestas e $O(k)$ enumeração de vizinhos, onde k é o grau do vértice.

2.2 Busca em Profundidade (DFS)

A busca em profundidade é um algoritmo de traversal que explora o grafo “o máximo possível” em cada ramo antes de retroceder.

Características:

- Usa uma **pilha** (LIFO)
- Complexidade: $O(V + E)$
- Não garante caminho mínimo
- Útil para detectar componentes conexas e ciclos

Pseudocódigo:

```
DFS(origem, destino):  
    stack.push(origem)  
    visited = {}  
    predecessor = {}  
  
    while stack not empty:  
        current = stack.pop()  
        if current in visited: continue  
        visited.add(current)  
  
        if current == destino:  
            return reconstructPath(predecessor, origem,  
                                    destino)  
  
        for each neighbor in adjacency[current]:  
            if neighbor not in visited:  
                predecessor[neighbor] = current  
                stack.push(neighbor)  
  
    return empty // nenhum caminho encontrado
```


2.3 Busca em Largura (BFS)

A busca em largura explora o grafo nível por nível, visitando todos os vizinhos a distância k antes de explorar distância $k + 1$.

Características:

- Usa uma **fila** (FIFO)
- Complexidade: $O(V + E)$
- **Garante caminho com menor número de arestas**
- Marca visitados ao *enfileirar*, não ao *desenfileirar*

Vantagem crítica: em grafos com ciclos, marcar visitados ao enfileirar previne que o mesmo vértice seja adicionado à fila múltiplas vezes.

2.4 Algoritmo de Dijkstra

O algoritmo de Dijkstra calcula o caminho de menor custo entre um nó de origem e todos os demais nós em um grafo com pesos não-negativos.

Características:

- Usa **fila de prioridade** (min-heap)
- Complexidade: $O(E \log V)$ com heap binário
- Retorna custo mínimo acumulado

2.4.1 Adaptação: Custo Inverso para Maximizar Dependência

Neste projeto, adaptamos Dijkstra para encontrar o caminho de **máxima dependência** de comunicação. A intuição é:

- Peso alto = muitos e-mails = relação forte
- Para favorecer arestas pesadas, convertemos: $\text{custo} = \frac{1}{\text{peso}}$
- Dijkstra minimiza custo $\Rightarrow$ prefere arestas pesadas

Exemplo:

- Aresta com peso 10: custo = 0.1
- Aresta com peso 1: custo = 1.0
- Dijkstra minimiza soma de custos $\Rightarrow$ escolhe aresta com peso 10

—

2.5 Tratamento de Ciclos

Todos os algoritmos implementados tratam ciclos explicitamente:

- **DFS**: conjunto `visited` previne revisitas
- **BFS**: marca visitados ao enfileirar
- **Dijkstra**: conjunto `settled` garante cada vértice é processado uma vez

3 Metodologia e Implementação

3.1 Arquitetura em Camadas

O projeto foi organizado em 7 pacotes (camadas):

1. **model**: classes de dados puros (Vertex, Edge, PathResult)
2. **graph**: estrutura do grafo (ContactGraph)
3. **service**: algoritmos (DFS, BFS, DistanceCalculator, CriticalPathService)
4. **parser**: leitura e parsing do dataset Enron
5. **util**: utilitários (validação de e-mail, fontes)
6. **design**: sistema de temas (cores, dark/light)
7. **view**: interface gráfica (Swing + GraphStream)

Esta separação segue os princípios SOLID:

- **SRP** (Single Responsibility): cada classe tem uma responsabilidade
- **OCP** (Open/Closed): extensível sem modificar código existente
- **DIP** (Dependency Inversion): depende de abstrações, não de implementações

—

3.2 Estrutura de Dados: Grafo com Lista de Adjacência

```
public class ContactGraph implements Serializable {
    private final Map<String, Vertex> vertexIndex; // email
        -> Vertex
    private final Map<Vertex, Map<Vertex, Edge>> adjacency;
        // origem -> (destino -> Edge)
    private final Set<String> ownerEmails; // senders (150
        usu\'arios Enron)
}
```

Operações:

- `addVertex(email)`: $O(1)$ médio (`HashMap.computeIfAbsent`)
- `addEdge(from, to)`: $O(1)$ médio; incrementa peso se existe
- `outDegree(v)`: $O(1)$ (tamanho do mapa interno)
- `inDegree(v)`: $O(V + E)$ (scan necessário)
- `vertexCount()`: $O(1)$
- `edgeCount()`: $O(V)$ (soma de tamanhos de mapas internos)

—

3.3 Algoritmos Implementados

3.3.1 1. Busca em Profundidade (DFS)

Classe: `DepthFirstSearch.java`

Método principal:

```
public PathResult search(ContactGraph graph, String  
    originEmail, String destinationEmail)
```

Detalhes de implementação:

- Usa `ArrayDeque` como pilha (evita `StackOverflowError`)
- Mantém conjunto `visited` para ciclos
- Mapa `predecessor` para reconstrução de caminho
- Retorna `PathResult` com lista de vértices ordenados

Garantias:

- Encontra caminho se existir (completo)
 - Não necessariamente o mais curto (unlike BFS)
 - Sem loops infinitos em ciclos
-

3.3.2 2. Busca em Largura (BFS)

Classe: `BreadthFirstSearch.java`

Método principal:

```
public PathResult search(ContactGraph graph, String  
    originEmail, String destinationEmail)
```

Detalhe crítico: marca visitados ao **enfileirar**, não ao desenfileirar.

```

queue.add(source);
visited.add(source); // <- Marca AQUI, nao ao dequeue
predecessor.put(source, null);

while (!queue.isEmpty()) {
    Vertex current = queue.poll();
    // ... processar
    for (Edge edge : graph.getOutEdges(current)) {
        Vertex neighbour = edge.getDestination();
        if (!visited.contains(neighbour)) {
            visited.add(neighbour); // <- Marca ao
            enfileirar
            predecessor.put(neighbour, current);
            queue.add(neighbour);
        }
    }
}
}

```

Garantia: retorna caminho com **menor número de arestas**.

—

3.3.3 3. Distância D (BFS por Níveis)

Classe: DistanceCalculator.java

Método principal:

```

public List<Vertex> getVerticesAtDistance(ContactGraph graph
    , String originEmail, int distance)

```

Lógica:

- Mantém lista de vértices em “nível atual”
- Para cada iteração de 1 a **distance**:
 - Expande todos vizinhos de saída do nível atual
 - Adiciona ao nível seguinte (se não visitado)
- Retorna apenas vértices do nível exato **distance**

Casos especiais:

- $D = 0$: retorna apenas o nó de origem
- Grafo esgotado antes de D : retorna lista vazia

—

3.3.4 4. Caminho Crítico (Dijkstra Adaptado)

Classe: `CriticalPathService.java`

Método principal:

```
public PathResult computeCriticalPath(ContactGraph graph,
    String originEmail, String destinationEmail)
```

Adaptação chave:

```
// custo inverso favorece arestas pesadas
double newCost = dist.get(current) + edge.getInverseCost();
    // 1.0 / weight
```

Saída:

- Lista de vértices no caminho
- Custo inverso total (métrica de Dijkstra)

- Dependência acumulada (soma dos pesos originais)
-

3.4 Leitura e Parsing do Dataset

Classe: `EnronDatasetReader.java`

Fluxo:

1. Escaneia diretório `maildir/` (150 usuários)
2. Para cada usuário, lê subpastas `sent/` e `_sent_mail/`
3. **Deduplicação:** calcula SHA-256 de cada arquivo
 - Se mesmo hash em `sent/` e `_sent_mail/`: processa apenas uma vez
 - Evita inflate artificial de pesos
4. Para cada arquivo único:
 - Parse com `EmailParser` (RFC 2822)
 - Extrai remetente (From:) e destinatários (To:)
 - Normaliza e-mails (lowercase, trim)
 - Adiciona aresta para cada destinatário
5. Serializa grafo em `graph.bin` para cache

Performance:

- Primeira execução: 1-3 minutos (scan completo + parse)
- Execuções subsequentes: < 1 segundo (carrega cache)
- Flag `-rebuild` força re-parse

4 Aprendizados e Conclusões

4.1 Conceitos Reforçados

4.1.1 Estruturas de Dados

- **Maps e Sets:** eficiência de lookup e membership testing
- **Listas vs. Deques:** LIFO (pilha) vs. FIFO (fila) e suas implicações
- **Filas de prioridade:** heap binário para Dijkstra eficiente
- **Representação de grafos:** adjacência vs. matriz vs. edge list

4.1.2 Algoritmos de Grafos

- **DFS e BFS:** diferenças semânticas e quando usar cada uma
- **Tratamento de ciclos:** importância de marcar visitados
- **Dijkstra:** otimização com fila de prioridade
- **Adaptações:** como modificar algoritmos para problemas específicos (custo inverso)

4.1.3 Engenharia de Software

- **SRP:** código mais testável e manutenível
- **Documentação:** JavaDoc não é overhead”, é comunicação
- **Cache:** serialização para performance (graph.bin)

- **Tratamento de erros:** validação robusta de entradas
-

4.2 Desafios Enfrentados

1. **Deduplicação:** como identificar e-mails duplicados entre pastas? **Solução:** SHA-256 hashing.
 2. **Performance:** parsing de 600 mil arquivos leva tempo. **Solução:** cache binário com Java Serialization.
 3. **Dijkstra adaptado:** como inverter a semântica de custo? **Solução:** aplicar transformação $\text{custo} = 1/\text{peso}$.
 4. **Ciclos:** evitar loops infinitos em grafos com ciclos. **Solução:** marcar visitados antes de explorar vizinhos.
-

4.3 Possíveis Extensões

- Implementar detecção de comunidades (ex: algoritmo de Louvain)
- Adicionar análise de centralidade (betweenness, closeness)
- Suportar grafos não-direcionados
- Exportar grafo em formato GraphML ou Gephi
- Paralelizar parsing com Java Stream parallelism
- Adicionar testes unitários (JUnit)

4.4 Conclusão

Este projeto foi uma aplicação prática e abrangente dos conceitos de estruturas de dados e algoritmos de grafos. Através da implementação, do zero, de DFS, BFS, cálculo de distância e Dijkstra adaptado, consolidei entendimento profundo sobre:

- Como diferentes estratégias de traversal afetam o resultado
- Importância de estruturas de dados eficientes (map vs. array)
- Trade-offs entre espaço e tempo
- Aplicação de conceitos teóricos em problemas reais

A arquitetura em camadas e a aderência aos princípios SOLID demonstraram que código bem estruturado é essencial não apenas para manutenibilidade, mas também para facilitar testes e validação.

A Diagramas UML

Diagrama de Classes

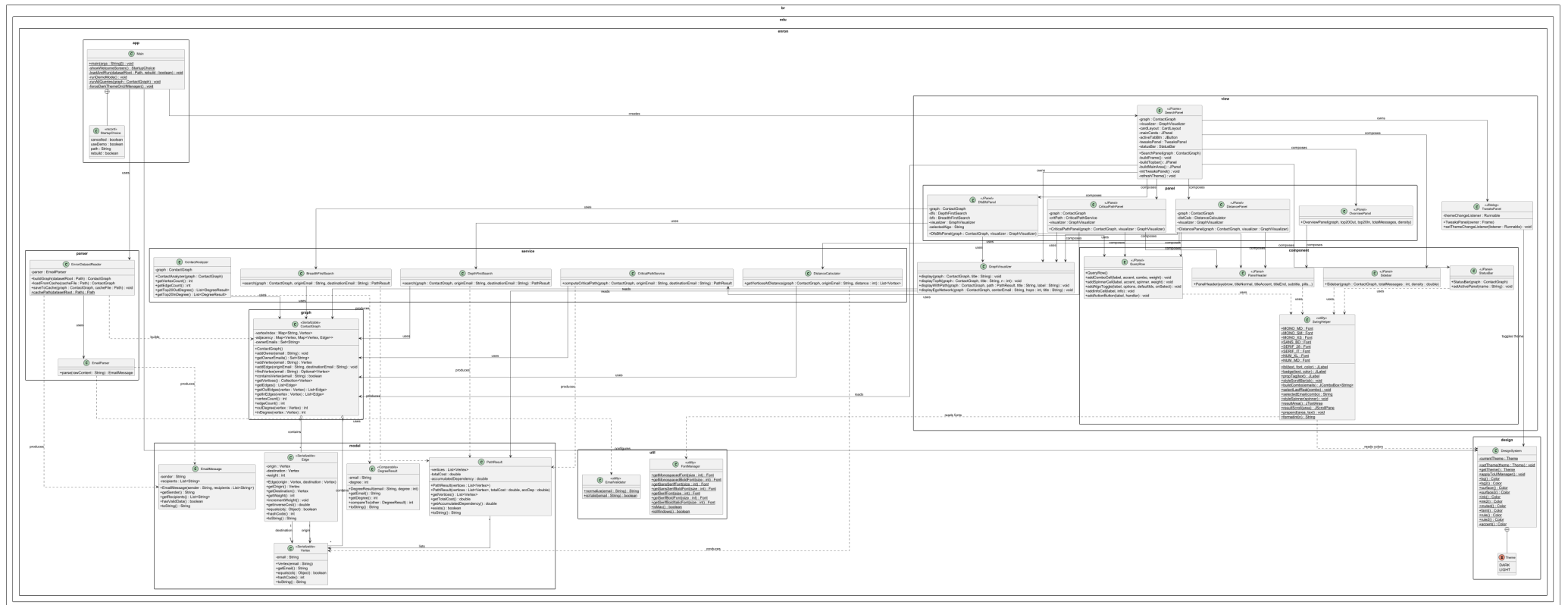


Figura 1 – Diagrama de Classes

Diagrama de Caso de Uso

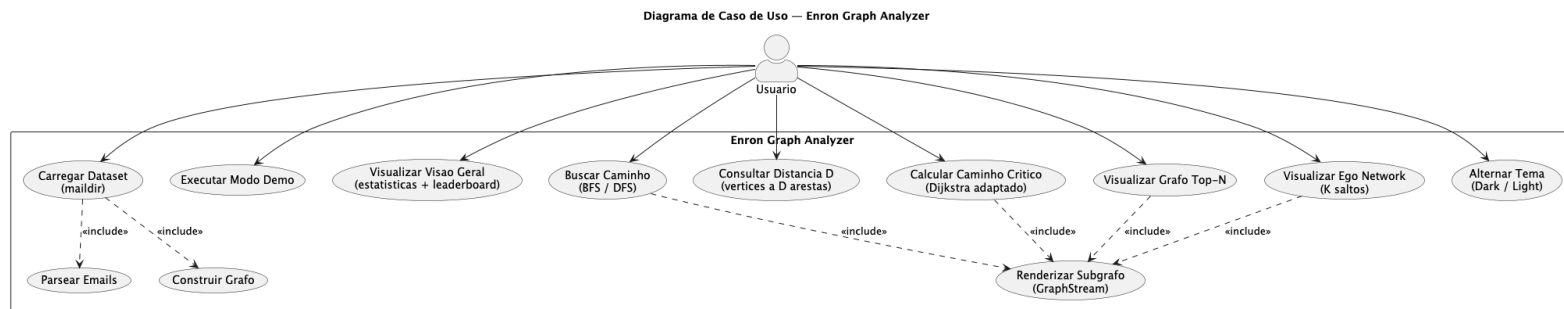


Figura 2 – Diagrama de Caso de Uso

Sequencia de Construcao do Grafo

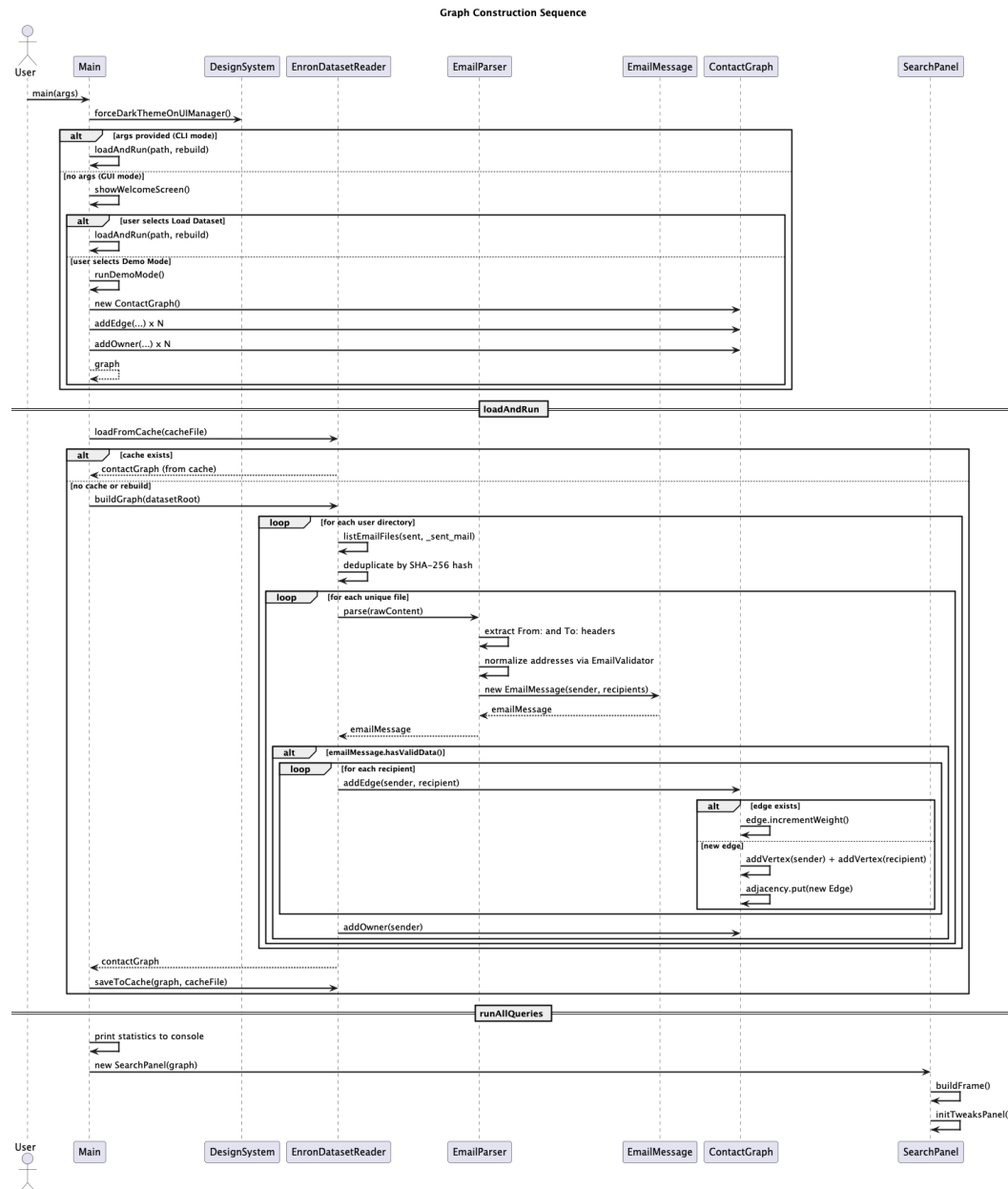


Figura 3 – Sequencia de Construcao do Grafo

Sequencia de Busca DFS e BFS

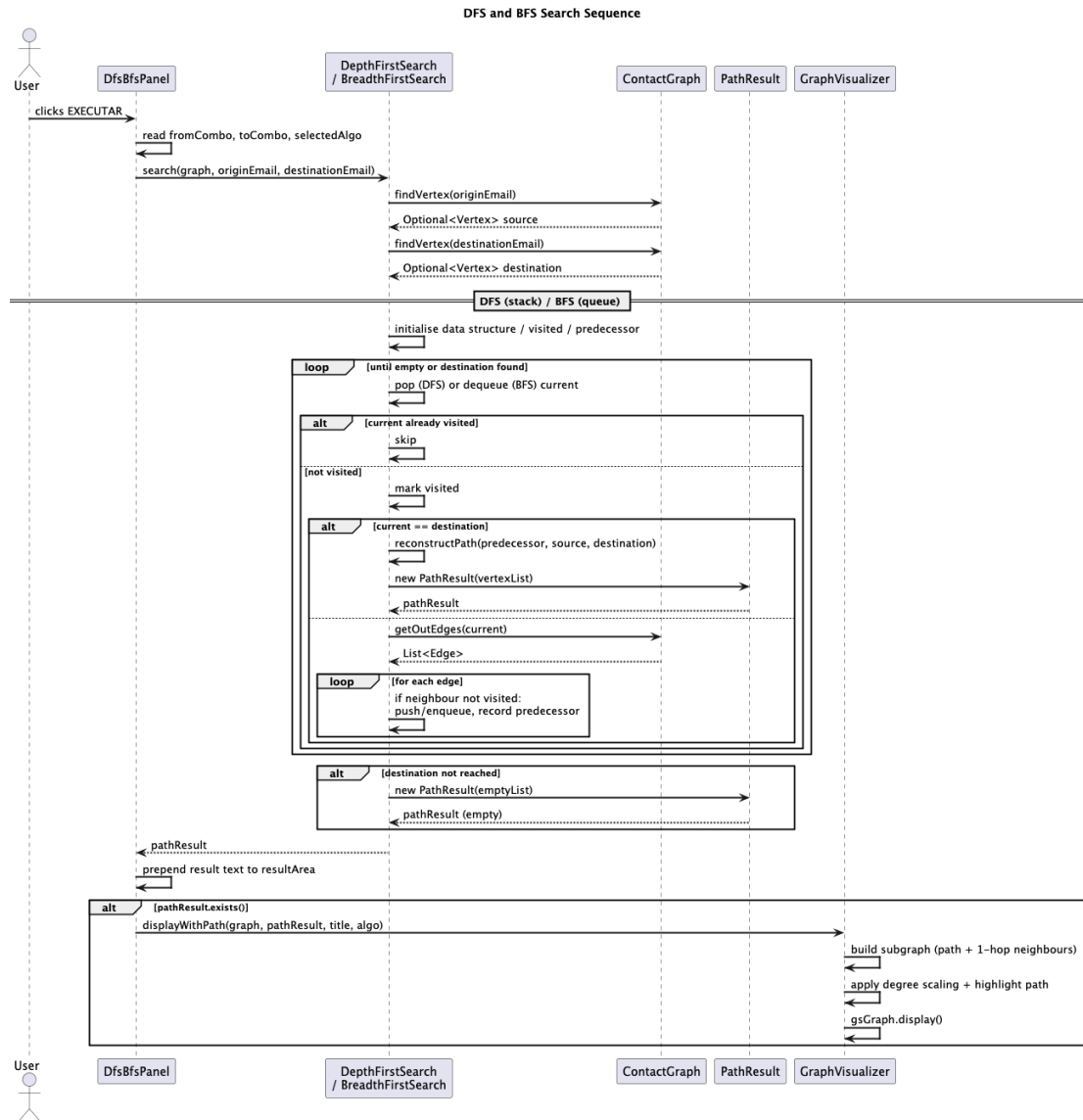


Figura 4 – Sequencia de Busca DFS e BFS

Sequencia de Caminho Critico

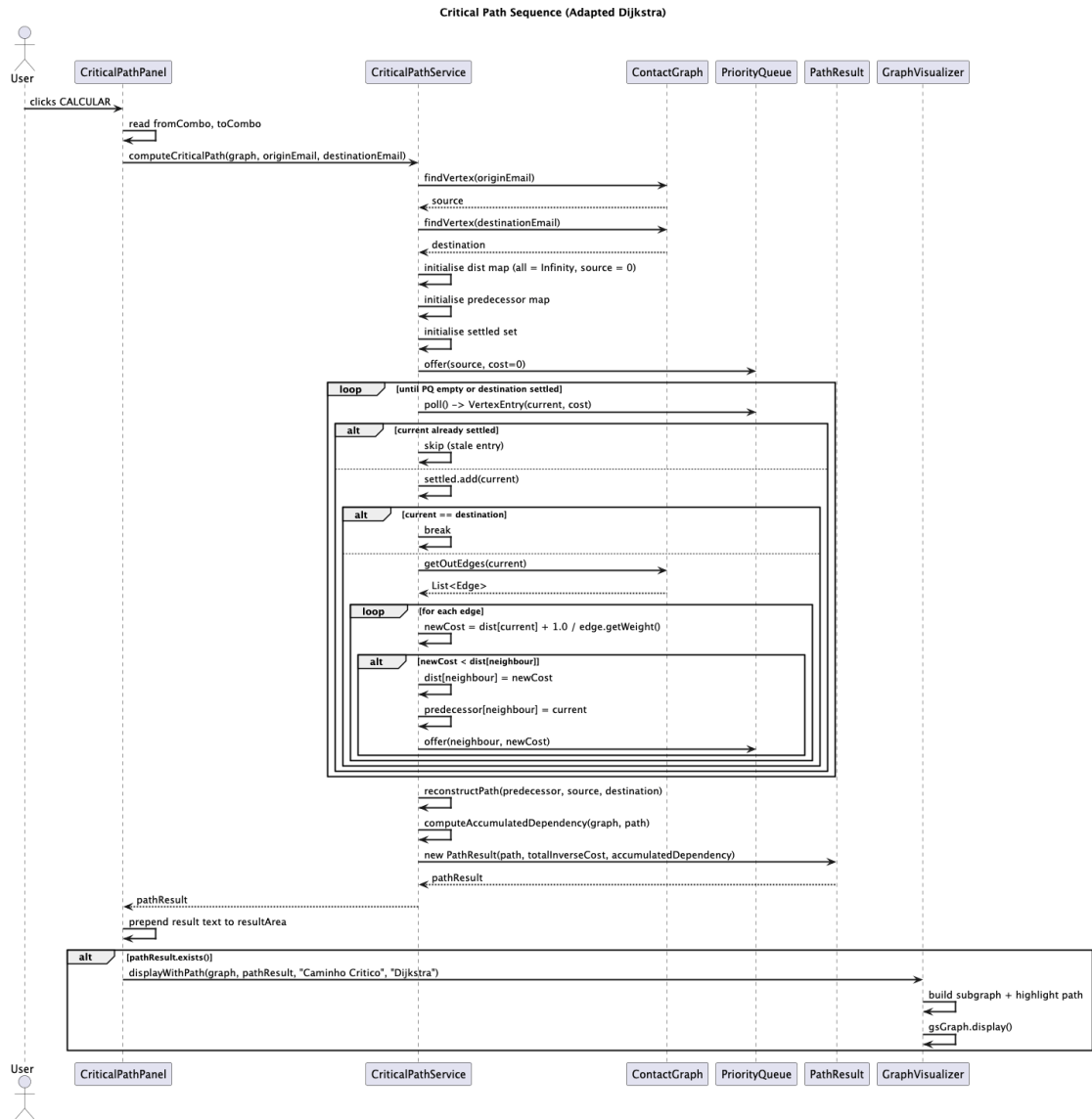


Figura 5 – Sequencia de Caminho Critico

Diagrama de Componentes

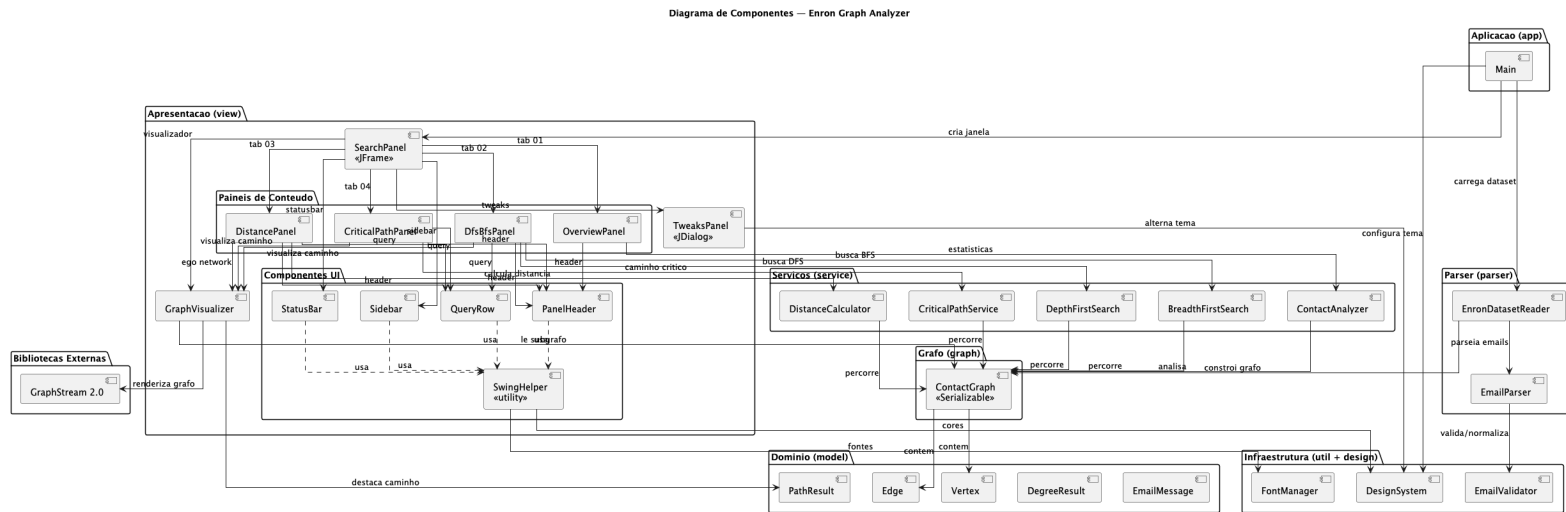


Figura 6 – Diagrama de Componentes

Diagrama de Dominio

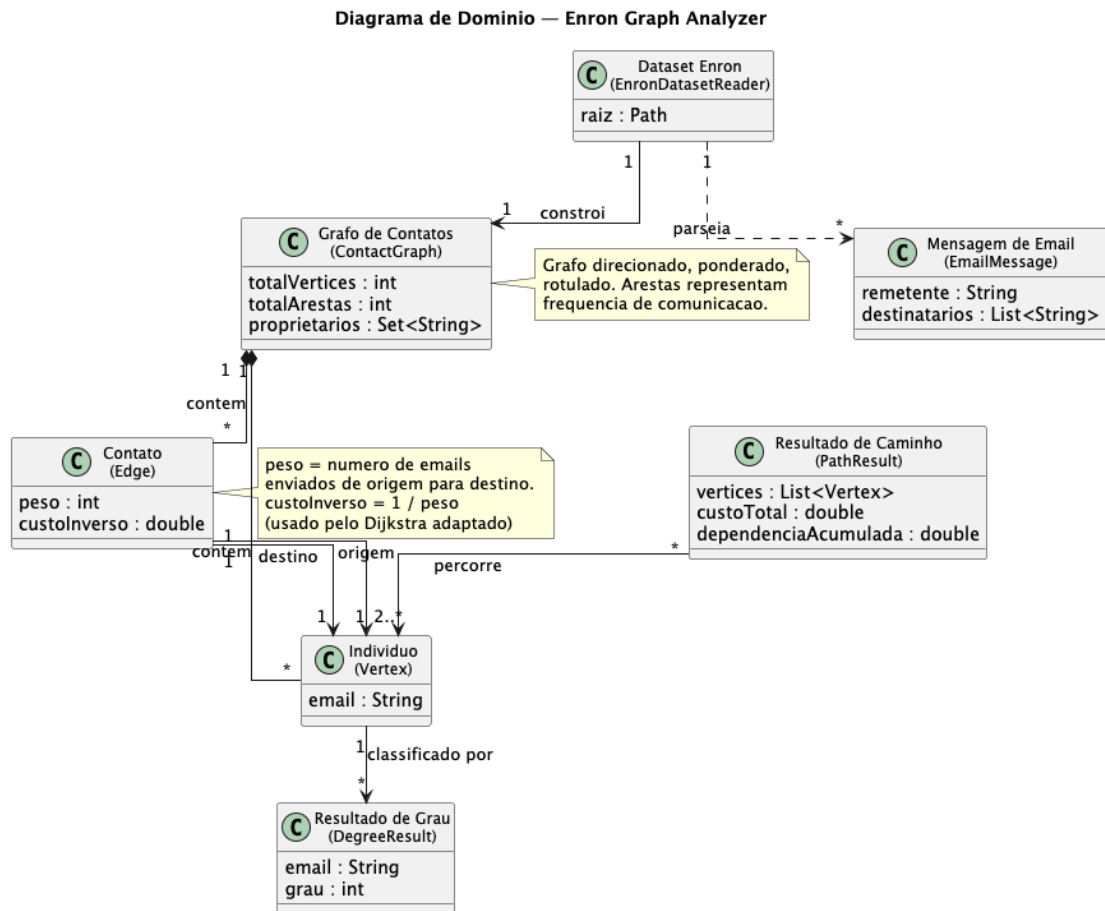


Figura 7 – Diagrama de Dominio

Diagrama de Atividade

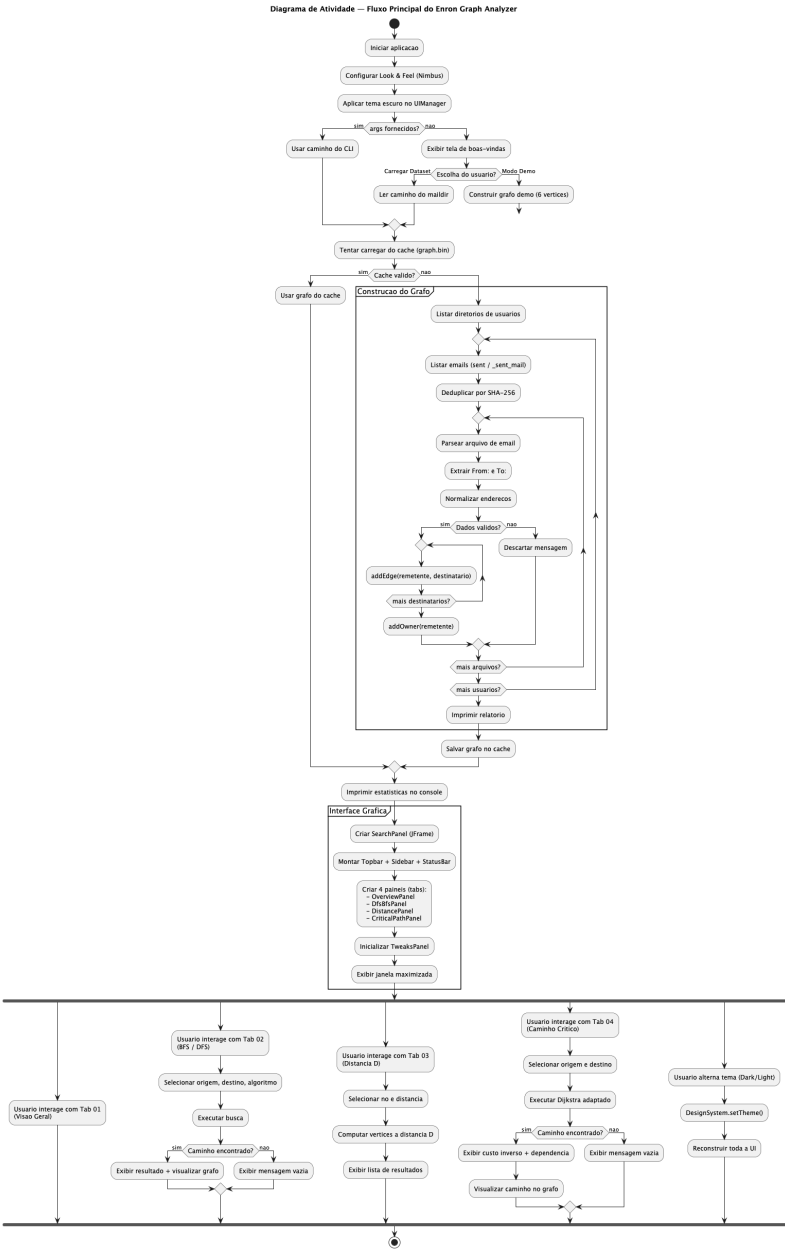


Figura 8 – Diagrama de Atividade

Diagrama de Estado

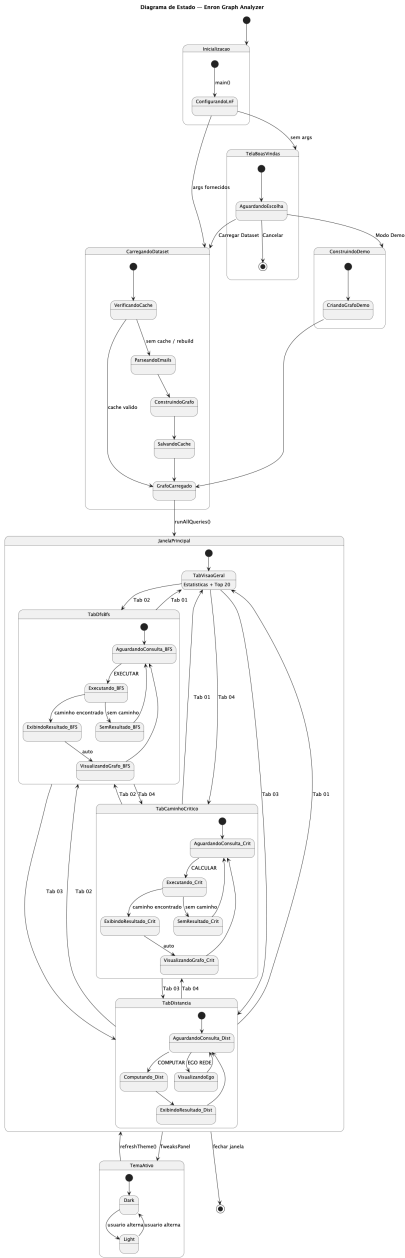


Figura 9 – Diagrama de Estado

Diagrama de Comunicacao

Diagrama de Comunicacao — Interacao Principal

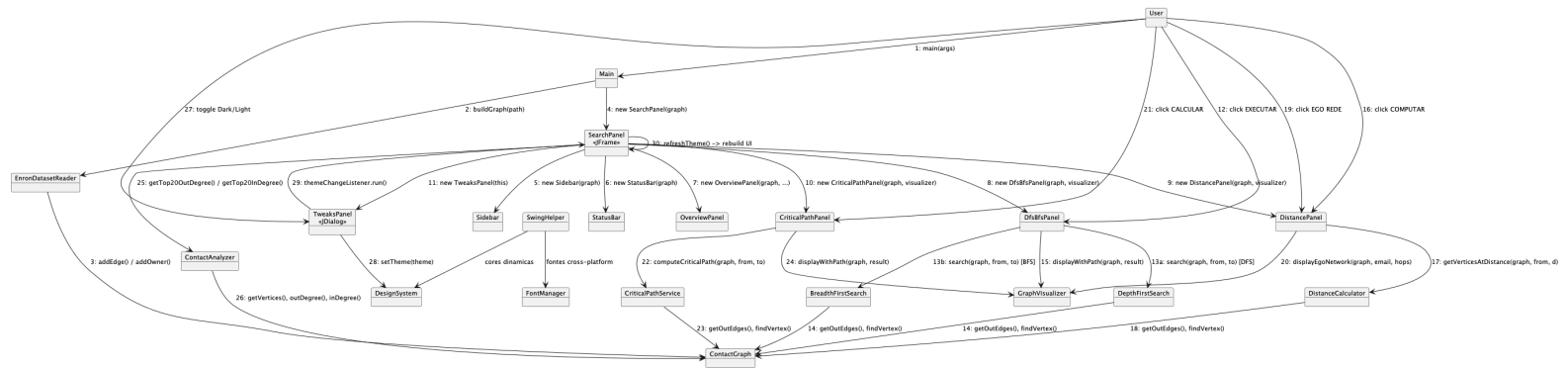


Figura 10 – Diagrama de Comunicacao

B Exemplos de Saída

B.1 Modo Demonstração

```
$ mvn exec:java

=== DEMO MODE ===

--- 1. GRAPH STATISTICS ---
  Vertices : 5
  Edges    : 6

--- 2. TOP 20 OUT-DEGREE (most active senders) ---
  1. alice@company.com (degree=2)
  2. bob@company.com   (degree=2)
  3. carol@company.com (degree=1)
  4. dave@company.com  (degree=1)

--- 3. TOP 20 IN-DEGREE (most contacted recipients) ---
  1. dave@company.com (degree=2)
  2. eve@company.com  (degree=2)
  3. bob@company.com  (degree=1)
  4. carol@company.com (degree=1)

--- 4-7. INTERACTIVE SEARCH PANEL ---
  Use the GUI windows to run DFS, BFS, Distance D and
  Critical Path.
```

[Janela Swing é exibida]

B.2 Consulta BFS no Demo

Origem: alice@company.com

Destino: eve@company.com

Algoritmo: BFS

RESULTADO:

alice@company.com -> bob@company.com -> eve@company.com

Comprimento: 2 arestas

Custo total das arestas: 2 mensagens (bob->eve tem peso 1)